Optimization in a Data Workflow

Concepts for the Mini-Project

DAEN 300 - Fall 2026



The dataset: FEMA IHP Valid Registrations

- After a declared disaster, households apply to FEMA's **Individuals and Households Program** (IHP) for help
- One row = one valid registration (an applicant household)
- Published on OpenFEMA; updated weekly
- All personal information removed; location at county / city / ZIP level

Scale: $\approx$ 25.5 million rows, several GB as CSV
$\Rightarrow$ too large for Excel; too large to handle carelessly in pandas

What is in a row?

Group	Example fields
Disaster	`disasterNumber`, `damagedStateAbbreviation`
Household	`ownRent`, `residenceType`, `grossIncome`
Inspection	`inspnIssued`, `inspnReturned`
Damage	`rpfvl`, `floodDamage`, `waterLevel`
Outcome	`repairAssistanceEligible`, `haAmount`

- `rpfvl`: FEMA-verified loss to the home's structure (floors, walls, electrical, plumbing, HVAC), in dollars
- `repairAssistanceEligible`: eligible for Home Repair Assistance (yes/no)

Three questions we can ask

1. Was the home **inspected**? *classification*
2. Is the household **eligible for repair assistance**? *classification*
3. How much **real-property damage** (`rpfvl`)? *regression*

Time matters: some fields are known at registration (home type, owner/renter, income); others exist only *after* inspection (damage, eligibility). Using "future" fields to predict outcomes is **data leakage**.

What does “optimize” mean here?

Make something measurably better — and show before vs. after.

	What we improve	Measure	Reading
1	Data pipeline	time, memory	PDSH 1–3
2	Code	run time	PDSH 1–2
3	Model	test score	PDSH 5
4	Decision	value gained	Week 13

Rule: *measure first, change one thing, measure again.*

1. Optimize the data pipeline

Idea: the fastest data is the data you never load.

- **At the source (API):**
  - `$select` — only the columns you need
  - `$filter` — only one disaster or one state
- **When loading (pandas):**
  - `usecols=` — skip unused columns
  - `dtype=` — `category` for codes (`ownRent`, `residenceType`), `bool` for flags
  - `chunksize=` — process a big file piece by piece
- **Measure:** `%timeit` for time, `df.memory_usage(deep=True)` for memory

Why data types matter

- A text column stores every string separately: `"Owner"`, `"Owner"`, `"Owner"`, ...
- A `category` column stores each label once, plus a small integer code per row
- 25 million rows $\times$ a few repeated labels $\Rightarrow$ large memory savings

Type	Stores	Typical use
`object` (text)	full string per row	free text
`category`	code per row + label list	repeated codes
`bool`	1 byte per row	yes/no flags
`float32`	4 bytes per row	dollar amounts

2. Optimize the code

Idea: find the slow part first; don't guess.

- **Profile** with `%prun`: which function takes the time?
- **Vectorize**: replace Python loops and `.apply()` with whole-column NumPy / pandas operations
- **Aggregate** with `groupby` instead of looping over groups
- **Report the speedup**: speedup $= t_{\text{before}} / t_{\text{after}}$

Why vectorization is faster: the loop runs in compiled C code over a contiguous array, not one Python object at a time.

3. Optimize the model

Idea: a model's settings (hyperparameters) are chosen by validation, not by default.

- **Classifier** for `repairAssistanceEligible`; **regressor** for `rpfvl`
- `GridSearchCV`: try combinations, score each with cross-validation
- **Validation curve**: training vs. validation score as complexity grows
  - too simple $\rightarrow$ underfit (both scores low)
  - too complex $\rightarrow$ overfit (gap between them)
- Always compare with a **baseline** (majority class, or the mean)

The shape of `rpfvl`

- Many applicants have **zero** verified damage
- The rest have a **long right tail**: a few very large losses
- Large values dominate squared-error metrics

Try: model $\log(1 + \texttt{rpfvl})$ with `np.log1p`, then convert back with `np.expm1`

- handles zeros ($\log 1 = 0$)
- compresses the tail, so errors become more like *relative* errors

4. Next year ... optimize a decision

Use the model's predictions to choose an action under a limit.

- Example: inspectors have limited hours — who should be inspected first?
- Each applicant has a **value** (from the model) and a **cost** (time)
- Choose the set with the most total value that fits the budget

$$\max \sum_{i \in S} v_i \quad \text{subject to} \quad \sum_{i \in S} w_i \leq W$$

- This is the **0/1 knapsack** problem $\rightarrow$ **dynamic programming** (Next semester!)

The full picture

$$\text{data prep} \rightarrow \text{ML} \rightarrow \text{optimization}$$

- Load less, store smaller
- Profile, then vectorize
- Tune with validation; compare with a baseline
- Turn predictions into a decision

In every step: **measure before, measure after, explain why.**